

Advanced programming tasks of OCaml

递归 - 语法

在 OCaml 编写递归函数时，需要额外标注 *rec* 关键字

```
let add a b =  
  a + b  
  
let rec factorial n =  
  if n = 1 then  
    1  
  else  
    n * factorial (n - 1)
```

递归 - 语法示例

给定一个数 x ，判断其是否为质数

```
let rec is_prime_helper x i =  
    if i == 1 then  
        true  
    else  
        (x mod i != 0) && (is_prime_helper x (i-1))  
  
let is_prime x =  
    is_prime_helper x (x-1)
```

递归 - 尾递归

递归调用在函数最后发生的递归，称为尾递归

```
let rec f1 () = f1 (); print_endline "Hello"  
let rec f2 () = print_endline "Hello"; f2 ()  
let rec f3 () = f3 ()
```

依次运行以上三个函数，会发现 f1 因栈溢出而中止，而 f2 和 f3 可以一直执行

这是因为 OCaml 编译器识别出尾递归特性，将其转换为迭代 (iteration)

递归 - 尾递归

```
let rec fact_rec n =  
  if n = 1 then  
    1  
  else  
    (n * fact_rec (n - 1)) mod 1000000007  
  
let rec fact_iter n result =  
  if n = 1 then  
    result  
  else  
    fact_iter (n - 1) ((result * n) mod 1000000007)
```

分别调用 fact_rec 999999 和 fact_iter 999999 1，只有前者会栈溢出

为什么要对 1000000007 取模？

练习 1

```
let rec is_prime_helper x i =  
  if i == 1 then  
    true  
  else  
    (x mod i != 0) && (is_prime_helper x (i-1))
```

- is_prime_helper 是否为尾递归？
- **假设**你无论如何也看不出 is_prime_helper 是不是尾递归，请设计一个实验来判断。

列表 - 语法

```
# [];;  
- : 'a list = []  
  
# [1; 2; 3];;  
- : int list = [1; 2; 3]  
  
# [false; true; false];;  
- : bool list = [false; true; false]  
  
# [[1; 2]; [3; 4]; [5; 6]];;  
- : int list list = [[1; 2]; [3; 4]; [5; 6]]
```

列表 - 拼接语法

:: 也叫 cons 操作符，将一个元素添加到一个列表前面，常见于函数式语言

@ 将两个列表拼接

```
# 1 :: [2; 3];;  
- : int list = [1; 2; 3]
```

```
# [1] @ [2; 3];;  
- : int list = [1; 2; 3]
```


列表 - 模式匹配语法

foo 和 list_max 分别匹配 int 和 'a list

```
let foo x =  
  match x with  
  | 1 -> 2  
  | 2 -> 3  
  | _ -> 0  
  
let rec list_max l =  
  match l with  
  | [] -> failwith "Empty list."  
  | [x] -> x  
  | h :: t -> max h (list_max t)
```

h 和 t 分别代表 head 和 tail

练习 2

```
let has_single_elem l =  
  match l with  
    (* todo *)  
  
is_empty_list [] (* false *)  
is_empty_list [1] (* true *)  
is_empty_list [2; 3; 4] (* false *)
```

实现 has_single_elem，判断 l 是否刚好包含一个元素

列表 - 模式匹配

计算列表长度的函数 length 的两个版本

```
let rec length_rec l =  
  match l with  
  | [] -> 0  
  | _ :: t -> 1 + length_rec t  
  
let rec length_iter sum l =  
  match l with  
  | [] -> sum  
  | _ :: t -> length_iter (sum + 1) t
```

尾递归函数有什么特点？

练习 3

```
let rec list_max l =  
  match l with  
  | [] -> failwith "Empty list."  
  | [x] -> x  
  | x :: xs -> max x (list_max xs)
```

将 list_max 改写成尾递归 list_max_iter (假设列表只包含正数)

```
let rec list_max_iter (* todo *) =  
  match l with  
  (* todo *)
```

列表 - 高阶函数 - map

接受函数作为参数的函数称为高阶函数

map 将 f 应用于 l 中所有元素，映射为另一个列表

```
let rec map f l =  
  match l with  
  | [] -> []  
  | h :: t -> f h :: map f t
```

上述函数等价于 List.map

练习 4

用 List.map 求一个列表所有元素的平方

列表 - 高阶函数 - filter

filter 会用谓词函数（predicate）检查列表中所有元素，只保留返回值为 true 的元素

```
List.filter (fun x -> x mod 2 = 0) [1; 2; 3; 4; 5; 6]  
- : int list = [2; 4; 6]
```

练习 5

实现一个 `filter : ('a -> bool) -> 'a list -> 'a list`

高阶函数

另一种高阶函数是将函数作为返回值

```
let const_func_factory x =  
  fun () -> x  
  
let always_return_5 = const_func_factory 5  
let always_return_42 = const_func_factory 42  
  
always_return_5 ();;  
always_return_42 ();;
```

const_func_factory 的返回值是一个函数，而两个生成的函数分别会返回 5 和 42

练习 6

实现 `count_123 : int list -> int`，返回列表中连续子序列 1 2 3 出现的次数

```
count_123 [1; 2] = 0
```

```
count_123 [1; 2; 3] = 1
```

```
count_123 [1; 2; 4; 3] = 0
```

```
count_123 [1; 2; 3; 1; 4; 2; 3; 1; 2; 3; 4] = 2
```


练习 7

实现尾递归函数 `sum_iter : int list -> int -> int`

```
sum_iter [2;3;4;5] 1 = 15
```

```
let many_ones = List.init 999999999 (Fun.const 1)
```

```
sum_iter many_ones 0 = 999999999
```

如果不是尾递归，对 `many_ones` 求和会导致栈溢出

练习 8

实现 $\text{diff} : \text{int list} \rightarrow \text{int list}$ ，求一个数列的差分

$\text{diff } [1;2;3;4;5] = [1;1;1;1]$

$\text{diff } [1;2;4;1;1] = [1;2;-3;0]$

$\text{diff } [1;0;1;0;1;0;0] = [-1;1;-1;1;-1;0]$

为了确保 $\text{diff} : \text{int list} \rightarrow \text{int list}$ ，你可以定义多个函数，

练习 9

实现 `fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`

```
fold_right (-) [1;3;4] 2
```

```
= 1 - (3 - (4 - 2)) = 0
```

```
List.fold_right (-) [1;3;4] 2
```

```
= 1 - (3 - (4 - 2)) = 0
```

```
List.fold_left (-) 2 [1;3;4]
```

```
= (((2 - 1) - 3) - 4) = -6
```

练习 10

现在有一个操作数组的函数 `smallbf`，接受两个数组 `mem : int list` 和 `cmd : string`，其中 `cmd` 只包含两个字符 `+` 和 `>`。`smallbf`操作的规则如下

- 最开始 `smallbf` 的指针指向数组的第 0 个元素
- `smallbf` 依次遍历 `cmd` 中的字符，对于每个字符 `c`
 - 如果 `c` 是 `+`，将指针处的数字加 1
 - 如果 `c` 是 `>`，将指针往后移动 1 位，如果指针已经指向最后一个元素，则将指针指向第 0 个元素
- `smallbf` 遍历完 `cmd` 之后，将修改后的 `mem` 返回

```
smallbf [0;0;0] "++>>+>++" = [4;0;1]
```

```
smallbf [3;0;-1;-2] "+>++++>>>>" = [4;5;-1;-2]
```

```
smallbf [42] ">>>>>>+>+>>" = [45]
```